

SUPPORT TICKET MANAGEMENT SYSTEM WITH AI-ASSISTED REPLY SUGGESTION

**Department of Computer Science
CHRIST (Deemed to be University)**

TITLE PAGE

SUPPORT TICKET MANAGEMENT SYSTEM WITH AI-ASSISTED REPLY SUGGESTION

A project report submitted in partial fulfilment of the requirements for the award of
Bachelor of Technology in Computer Science and Engineering

Submitted by
[Student Name(s)]
Register Number(s)

Under the guidance of
[Guide Name]
Designation

**Department of Computer Science
CHRIST (Deemed to be University)
Bengaluru – 560029
Month Year**

CERTIFICATE

This is to certify that the project entitled “**Support Ticket Management System with AI-Assisted Reply Suggestion**” submitted by **[Student Name(s)]** bearing Register No. **[Register Number(s)]** in partial fulfilment of the requirements for the award of **Bachelor of Technology in Computer Science and Engineering** by CHRIST (Deemed to be University) is a record of bonafide work carried out by them under my supervision.

The matter embodied in this project report has not been submitted earlier for the award of any other degree/diploma to the best of my knowledge.

Guide Name
Designation
Department of Computer Science
CHRIST (Deemed to be University)

Place: Bengaluru
Date:

ACKNOWLEDGMENTS

We thank our project guide, **[Guide Name]**, for continuous guidance and support throughout the project. We also thank the Department of Computer Science, CHRIST (Deemed to be University), for providing the necessary resources and lab facilities. We acknowledge all faculty and friends who contributed to the successful completion of this project.

ABSTRACT

Support teams in organisations handle large volumes of customer queries via email and other channels. Managing these as tickets, tracking conversations, and replying consistently is time-consuming and error-prone when done manually. This project designs and implements a Support Ticket Management System that integrates email-based ticket creation, customer identification by domain, role-based access for agents and admins, and AI-assisted reply suggestion. The system uses a Flask-based web application with MongoDB for persistence. Incoming emails are polled via IMAP, parsed for subject and body, and matched to companies and contacts by sender domain; new tickets are created or existing threads are updated using Message-ID, In-Reply-To, References, and Gmail thread ID. Agents view tickets in a web dashboard, filter by status and domain, and reply to customers; replies are sent via SMTP with proper threading and stored in the same ticket. Reply suggestion is implemented in three tiers: reuse of support replies from similar past tickets using text similarity, optional Google Gemini-based generation when an API key is configured, and fallback to issue-type templates. An optional Phase 2 module provides ticket classification (issue type, priority, sentiment) using TF-IDF and XGBoost, trained on a support-ticket dataset; when available, these predictions enrich the context for Gemini. The system demonstrates end-to-end flow from email to ticket to reply, with clear separation of email processing, web application, and optional machine-learning components. Major outcomes include automated ticket creation and threading, customer and domain tagging, consistent reply quality through suggestions, and a scalable architecture for future enhancements such as more channels and advanced analytics.

TABLE OF CONTENTS

- 1. Introduction ... 1
 - 1.1. Project Description
 - 1.2. Existing System
 - 1.3. Objectives
 - 1.4. Purpose, Scope and Applicability
 - 1.4.1. Purpose

1.4.2.	Scope
1.4.3.	Applicability
1.5.	Overview of the Report
2.	System Analysis and Requirements ...
2.1.	Problem Definition
2.2.	Requirements Specification
2.3.	Block Diagram
2.4.	System Requirements
2.4.1.	User Characteristics
2.4.2.	Software and Hardware Requirements
2.4.3.	Constraints
2.5.	Conceptual Models
2.5.1.	Data Flow Diagram
2.5.2.	ER Diagram
3.	System Design ...
3.1.	System Architecture
3.2.	Module Design
3.3.	Database Design
3.3.1.	Tables and Relationships
3.3.2.	Data Integrity and Constraints
3.4.	Interface Design and Procedural Design
3.5.	Reports Design
4.	Implementation ...
4.1.	Implementation Approaches
4.2.	Coding Standard
4.3.	Coding Details
4.4.	Screen Shots
5.	Testing ...
5.1.	Test Cases
5.2.	Testing Approaches
5.3.	Test Reports
6.	Conclusion ...
6.1.	Design and Implementation Issues
6.2.	Advantages and Limitations
6.3.	Future Scope of the Project
	Appendices
	References

LIST OF TABLES

Table No.	Title	Page No.
2.1	Functional Requirements	
2.2	Software Requirements	
2.3	Hardware Requirements	
3.1	Customers Collection Schema	
3.2	Tickets Collection Schema	
3.3	Agents / Admins Schema	
5.1	Login Test Cases	
5.2	Ticket List and Filter Test Cases	
5.3	Reply Suggestion Test Cases	

LIST OF FIGURES

Fig. No.	Figure Name	Page No.
2.1	High-level Block Diagram	
2.2	Context Diagram (DFD Level 0)	
2.3	Data Flow Diagram Level 1	
2.4	Entity-Relationship Diagram	
3.1	System Architecture (3-Tier)	
3.2	Module Dependency Diagram	
3.3	Application Flow (Reply Suggestion)	
4.1	Login Screen	
4.2	Ticket List Screen	
4.3	Ticket Detail and Reply Screen	

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
ER	Entity-Relationship
IMAP	Internet Message Access Protocol
ML	Machine Learning
SMTP	Simple Mail Transfer Protocol
TF-IDF	Term Frequency–Inverse Document Frequency
TOC	Table of Contents
UI	User Interface

Abbreviation	Expansion
XGBoost	Extreme Gradient Boosting

1. INTRODUCTION

This chapter summarises the project background, existing systems, objectives, purpose, scope, applicability, and the organisation of the report.

1.1. PROJECT DESCRIPTION

Customer support teams receive large numbers of requests through email and other channels. Without a structured system, requests are hard to track, conversations get fragmented, and responses can be slow or inconsistent. This project develops a **Support Ticket Management System** that (i) turns incoming emails into tickets and threads them correctly, (ii) identifies customers and companies by email domain, (iii) provides a web dashboard for agents and admins to view, filter, and manage tickets, (iv) allows agents to send replies by email while keeping the full conversation in one place, and (v) suggests reply text using similar past tickets, an optional AI (Gemini) service, or predefined templates. The system uses a Python Flask web application, MongoDB for storage, and an email-processing pipeline (IMAP/SMTP). An optional machine-learning module (Phase 2) classifies tickets by issue type, priority, and sentiment using TF-IDF and XGBoost, improving context for AI-generated suggestions. The project addresses the need for a single, scalable platform that reduces manual effort and improves response quality in support workflows.

1.2. EXISTING SYSTEM

Existing approaches include (i) generic email clients with folders and labels, (ii) shared inboxes with manual assignment, and (iii) commercial help-desk tools. Email-only workflows lack centralised ticket state, threading by business identity (company/contact), and reply assistance. Shared inboxes do not always preserve thread identity or link replies to a unique ticket. Commercial tools are feature-rich but can be costly and complex. Many small or mid-size teams need a lightweight system that automates ticket creation from email, tags customers by domain, and assists agents with reply suggestions without full enterprise licensing. This project fills that gap with an integrated, email-driven ticket system and AI-assisted reply suggestion.

1.3. OBJECTIVES

- To design and implement an email-integrated support ticket system that creates and threads tickets from incoming emails and links them to companies and

contacts by domain.

- To provide a web-based dashboard for agents and admins to view, filter (by status and domain), and manage tickets and to send replies that are emailed to the customer and stored in the same ticket.
- To implement a three-tier reply suggestion: similar-ticket reuse, optional Gemini-based generation, and issue-type template fallback, with optional ML-based classification (issue, priority, sentiment) to enrich context.

1.4. PURPOSE, SCOPE AND APPLICABILITY

1.4.1. Purpose

The project aims to (i) reduce manual ticket creation and threading, (ii) improve traceability of customer conversations by company and domain, (iii) speed up agent responses through reply suggestions, and (iv) demonstrate integration of email, web application, and optional ML/AI components in a single system.

1.4.2. Scope

The scope includes: email polling (IMAP) and parsing; customer/company resolution by sender domain; ticket creation and threading (Message-ID, In-Reply-To, References, thread ID); Flask web app with login, ticket list, ticket detail, manual ticket creation, and reply submission; SMTP sending with proper threading; reply suggestion (similar tickets, Gemini, templates); optional Phase 2 classifier for issue/priority/sentiment; MongoDB for customers, tickets, agents, and admins. Assumptions: use of Gmail-compatible IMAP/SMTP, MongoDB available, and optional Gemini API key for AI suggestions. Out of scope: multi-channel ingestion beyond email, advanced workflow automation, and full ITSM features.

1.4.3. Applicability

The system is directly applicable to small and medium support teams that handle email-based support. It can be extended to other channels and integrated with CRM or help-desk systems. Indirectly, it demonstrates patterns for email-to-database pipelines, role-based web dashboards, and AI-assisted text generation in customer service.

1.5. OVERVIEW OF THE REPORT

Chapter 2 presents system analysis and requirements: problem definition, functional and non-functional requirements, block diagram, user characteristics, software and hardware requirements, constraints, and conceptual models (DFD and ER). **Chapter 3** describes system design: architecture, modules, database design, interface and procedural design, and reports. **Chapter 4** covers implementation: approaches, coding standards, key code details, and screen shots. **Chapter 5** documents testing: test cases, approaches, and test reports. **Chapter 6** concludes with design and implementation

issues, advantages and limitations, and future scope. Appendices provide user manual and supplementary material; references follow.

2. SYSTEM ANALYSIS AND REQUIREMENTS

This chapter defines the problem, specifies requirements, presents the block diagram and system requirements, and describes conceptual models (DFD and ER).

2.1. PROBLEM DEFINITION

The problem is to support customer support teams that receive many email requests by (a) automatically creating tickets from emails and threading follow-up messages to the correct ticket, (b) identifying and tagging customers and companies by email domain, (c) giving agents a single place to view and reply to tickets with replies sent by email and stored in the ticket, and (d) suggesting reply text to reduce time and keep quality consistent. Sub-problems addressed include: reliable email parsing and thread matching (Message-ID, In-Reply-To, References, thread ID); mapping sender email to company and contact; secure, role-based web access; sending replies that preserve email thread; and generating useful suggestions when similar past replies exist, when an AI API is available, or from templates.

2.2. REQUIREMENTS SPECIFICATION

Functional requirements: (1) Poll inbox via IMAP and process new messages. (2) Parse subject, body, From, Message-ID, In-Reply-To, References. (3) Resolve or create company and contact by sender domain; create or update ticket and store comment. (4) Send acknowledgment email for new tickets (optional). (5) Web login for agents and admins (email/password). (6) List tickets with sort by updated time; filter by status and domain. (7) View ticket detail with full conversation. (8) Create ticket manually (customer, subject, issue, priority, body). (9) Submit reply; send via SMTP with threading and store in ticket. (10) Suggest reply (similar ticket, Gemini, or template). (11) Update ticket status (Open, Pending, Closed). (12) Optional: classify ticket with ML (issue, priority, sentiment) for suggestion context.

Non-functional requirements: Availability of web app and email pipeline; security via password hashing and session management; scalability via database indexing; maintainability through modular code and configuration (e.g. .env).

ID	Requirement	Type
FR1	Email polling and parsing	Functional
FR2	Customer/company resolution by domain	Functional
FR3	Ticket create/thread and comment storage	Functional
FR4	Optional acknowledgment email	Functional

ID	Requirement	Type
FR5	Web login (agent/admin)	Functional
FR6	Ticket list and filters	Functional
FR7	Ticket detail and conversation	Functional
FR8	Manual ticket creation	Functional
FR9	Reply send and store	Functional
FR10	Reply suggestion (similar/Gemini/template)	Functional
FR11	Status update	Functional
NFR1	Secure authentication	Non-functional
NFR2	Responsive UI	Non-functional

Table 2.1 — Functional and key non-functional requirements

2.3. BLOCK DIAGRAM

The system is decomposed into seven major blocks that work together to deliver end-to-end ticket management and reply assistance.

Block 1 — Email Gateway: This component is responsible for connecting to the mail server via IMAP (e.g. IMAP over SSL to Gmail), fetching only UNSEEN messages to avoid reprocessing, and parsing each message to extract the subject, plain-text body, From header, Message-ID, In-Reply-To, References, and (where available) Gmail thread ID (X-GM-THRID). The parsed output is a structured object (e.g. subject, from, body, message_id, in_reply_to, references, thruid, uid) that is passed to the next block. Attachment metadata may also be extracted for future use.

Block 2 — Customer Resolution: The customer resolution block takes the sender email address from the parsed email and derives the domain (the part after the @ symbol). It queries the customers collection for a document whose domain matches. If no company exists, it creates a new company document with a generated ID (e.g. CMP-A001) and embeds the first contact with a generated contact ID (e.g. CUS-A001). If the company exists but the sender email is not in the contacts list, it appends a new contact. The block returns company_id and customer_id for use by the ticket manager.

Block 3 — Ticket Manager: The ticket manager determines whether the incoming email belongs to an existing ticket or should create a new one. It first checks In-Reply-To and References: if any of these message IDs match a comment's message_id or the ticket's ack_message_id, the ticket is found. Next it checks the Gmail thread ID if present. Then it looks for a ticket ID pattern in the subject (e.g. [TCK-0001]). If none of these match, it generates a new ticket ID (e.g. TCK-0002) and creates a new ticket document. It then either inserts the new ticket with the first comment or appends the comment to the existing ticket, and optionally triggers the sending of an acknowledgment email for new tickets.

Block 4 — Database: MongoDB holds four main collections: customers (companies and their contacts), tickets (with embedded comments), agents, and admins. All persis-

tent state for the application and the email pipeline is stored here. Indexes on `thread_id`, `company_id`, `customer_id`, and `comments.message_id` support fast lookups for threading and filtering.

Block 5 — Web Application: The web application is a Flask server that serves the login page, ticket list (with status and domain filters), ticket detail (conversation and reply form), and manual ticket creation. It uses session-based authentication (admin and agent roles), and all ticket and reply operations read from and write to the database. The reply suggestion endpoint is invoked asynchronously (e.g. via `fetch`) and returns JSON.

Block 6 — Reply Suggestion: This block implements a three-tier strategy. First it searches for similar past tickets (using text similarity on subject and customer messages) and returns the best support reply if the score exceeds a threshold. If that is not used or disabled, it optionally enriches the conversation with Phase 2 predictions (issue, priority, sentiment) and calls the Google Gemini API to generate a reply. If the API is unavailable or not configured, it falls back to issue-type-based templates. The result is returned to the front end for the agent to edit or send as-is.

Block 7 — Optional ML Module (Phase 2): When model files are present, this module loads a TF-IDF vectorizer and three XGBoost classifiers (issue type, priority, sentiment). Given the conversation text, it returns predicted issue type, priority, and sentiment, which can be passed to the reply suggestion block to improve Gemini context.

Draw a block diagram with these seven blocks and arrows indicating data flow: Email → Email Gateway → Customer Resolution → Ticket Manager → Database; Web Application ↔ Database; Web Application → Reply Suggestion (↔ Database for similar-ticket search, ↔ Gemini API and Template logic); Reply Suggestion optionally ← ML Module.

Fig. 2.1 — High-level block diagram of the support ticket system

2.4. SYSTEM REQUIREMENTS

2.4.1. User Characteristics

- **Agents:** Support staff who view tickets, filter by status/domain, open ticket detail, use suggested or custom replies, send replies, and update status.
- **Admins:** Same as agents; may manage users and system settings in an extended version.
- **End customers:** Send emails that become tickets; receive replies by email; no direct use of the web UI.

2.4.2. Software and Hardware Requirements

Item	Description
OS	Windows 10/11, macOS, or Linux
Runtime	Python 3.9+
Web framework	Flask 3.0+
Database driver	PyMongo 4.0+
Libraries	python-dotenv, google-generativeai (optional), werkzeug
Email	IMAP/SMTP (Gmail-compatible)
Browser	Chrome, Firefox, Edge (latest)
Optional (Phase 2)	pandas, scikit-learn, xgboost

Table 2.2 — Software requirements

Item	Description
Processor	2+ cores recommended
RAM	4 GB minimum, 8 GB recommended
Disk	500 MB for app and models
Network	Internet for email and optional Gemini API

Table 2.3 — Hardware requirements

2.4.3. Constraints

Constraints include: dependency on IMAP/SMTP and provider limits (e.g. Gmail); MongoDB must be reachable; Gemini suggestion requires valid API key and quota; Phase 2 ML requires training data and disk for model files; single-instance deployment assumed unless scaled out.

2.5. CONCEPTUAL MODELS

2.5.1. Data Flow Diagram

Level 0 (Context): External entities: Customer (sends email), Agent/Admin (uses web). System: Support Ticket System. Flows: Customer → Email in; System → Reply email out; Agent → Login/requests; System → Ticket list/detail/reply/status.

Level 1: Processes: 1) Fetch & Parse Email, 2) Resolve Customer, 3) Create/Update Ticket, 4) Send Ack (optional), 5) Authenticate User, 6) List/Filter Tickets, 7) View Ticket & Suggest Reply, 8) Send Reply & Update Ticket, 9) Update Status. Data stores: Customers, Tickets, Agents, Admins. Show flows between processes and stores (e.g. parsed email → Resolve Customer → Customers; Resolve Customer + Create/Update Ticket → Tickets).

Fig. 2.2 — Context diagram (DFD Level 0)

Fig. 2.3 — Data flow diagram Level 1

2.5.2. ER Diagram

Entities and attributes:

- **Customer (Company):** company_id (PK), company_name, domain (unique), address, created_at. The company represents an organisation identified by its email domain (e.g. example.com). Each company document contains an embedded array of contacts.
- **Contact:** Within a company document, each contact has customer_id (unique within that company), name, email, contact (phone or other), agent_id (optional assigned agent), created_at. A contact is a person from that company who has sent at least one email; the same person may have multiple tickets over time.
- **Ticket:** ticket_id (PK), thread_id (email thread identifier where available), company_id (FK to Customer), customer_id (FK to Contact within that company), subject, issue (e.g. Technical, Billing, Account), status (Open, Pending, Closed), priority (High, Medium, Low), created_at, updated_at, ack_message_id (Message-ID of the automatic acknowledgment email, used for threading). The ticket document embeds an array of comments.
- **Comment:** Each comment has comment_id, message_id (email Message-ID for threading), body (plain text), created_at, from_support (boolean). Comments are stored in chronological order within the ticket; customer messages have from_support false, support replies have from_support true.
- **Agent:** agent_id (PK), name, email (unique), contact, department, timeslot, password_hash, permissions, is_active, created_at. Agents log in to the web application and perform ticket viewing, reply, and status updates.
- **Admin:** admin_id (PK), name, email (unique), contact, password_hash, permissions, is_active, created_at. Admins have the same capabilities as agents in the base system; the role distinction allows future extension (e.g. user management, settings).

Relationships:

- A Company has one or many Contacts (1:N).
- A Ticket belongs to exactly one Company and exactly one Contact (N:1 to Company, N:1 to Contact).
- A Ticket contains zero or many Comments (1:N, embedded).
- Agents and Admins are independent entities used only for authentication and authorisation; no foreign key from Ticket to Agent in the current design (assignment can be added later).

Draw the ER diagram with entities as rectangles, attributes listed inside, and relationships as diamonds or directed edges with cardinalities (1, N) as per the above. Use a drawing canvas as per project guidelines.

Fig. 2.4 — Entity-relationship diagram

3. SYSTEM DESIGN

This chapter describes the system architecture, module design, database design, interface and procedural design, and reports design.

3.1. SYSTEM ARCHITECTURE

The system follows a **three-tier architecture**. (1) **Presentation tier**: Flask-rendered HTML (Jinja2) with CSS; pages for login, ticket list, ticket detail, and ticket create. (2) **Application tier**: Flask routes and business logic—email pipeline (separate process or script), customer resolution, ticket create/update, reply suggestion (similar ticket, Gemini, template), optional ML prediction, authentication and session handling. (3) **Data tier**: MongoDB collections (customers, tickets, agents, admins). The email pipeline can run as a separate process that polls IMAP and writes to MongoDB; the web app and pipeline share the same database. Reply suggestion uses in-process similarity search, optional external Gemini API, and local template logic; Phase 2 ML loads local model files when present.

Fig. 3.1 — Three-tier system architecture

3.2. MODULE DESIGN

- **Email gateway module**: IMAP connect, fetch UNSEEN, parse message (subject, body, From, Message-ID, In-Reply-To, References, thread ID), extract sender email, pass to customer resolution and ticket manager; mark seen after processing.
- **Customer resolution module**: Given sender email, derive domain; find or create company (by domain); find or create contact (by email); return `company_id` and `customer_id`.
- **Ticket manager module**: Resolve or create ticket (by In-Reply-To/References/thread ID/subject ticket ID); persist comment; for new ticket, optionally send ack and store `ack_message_id`.
- **Web auth module**: Login (email/password), lookup in admins then agents, verify password hash, set session (`user_id`, `role`); logout clear session; `login_required` decorator.
- **Ticket list module**: Query tickets by optional status and domain filter, sort by `updated_at`, attach customer info per ticket, render list with filters.
- **Ticket detail module**: Load ticket, sort comments by time, resolve customer info, render conversation and reply form.
- **Reply suggestion module**: Build conversation text; find similar tickets (text

similarity over subject + customer messages), return best support reply; else call Gemini with optional ML-enriched context; else return template by issue type.

- **Reply send module:** Get customer email, build In-Reply-To/References from last comment, send SMTP, store support comment and updated_at.
- **Status update module:** Validate status (Open/Pending/Closed), update ticket and updated_at.
- **Phase 2 ML module (optional):** Load vectorizer and XGBoost models; predict issue, priority, sentiment for conversation text; return dict for suggestion context.

Fig. 3.2 — Module dependency diagram

3.3. DATABASE DESIGN

MongoDB is used with four main collections.

3.3.1. Tables and Relationships

Customers: _id (e.g. CMP-A001), company_name, domain, address, created_at, contacts (array of { customer_id, name, email, contact, agent_id, created_at }). One document per company; contacts embedded.

Tickets: _id (e.g. TCK-0001), thread_id, company_id, customer_id, subject, issue, status, priority, created_at, updated_at, ack_message_id (optional), comments (array of { comment_id, message_id, body, created_at, from_support }). One document per ticket; comments embedded.

Agents: _id, name, email, contact, department, password_hash, is_active, created_at, etc.

Admins: _id, name, email, contact, password_hash, is_active, created_at, etc.

Relationships: tickets.company_id references customers._id; tickets.customer_id references a contact's customer_id within a customer document. No formal FK in MongoDB; application ensures consistency.

Field	Type	Description
_id	string	Company ID (e.g. CMP-A001)
company_name	string	Display name
domain	string	Email domain (unique)
contacts	array	{ customer_id, name, email, ... }

Table 3.1 — Customers collection schema (main fields)

Field	Type	Description
_id	string	Ticket ID (e.g. TCK-0001)
company_id	string	Reference to customers._id
customer_id	string	Contact id in company
subject	string	Ticket subject
issue	string	Issue type
status	string	Open, Pending, Closed
priority	string	High, Medium, Low
comments	array	{ comment_id, message_id, body, from_support, created_at }

Table 3.2 — Tickets collection schema (main fields)

3.3.2. Data Integrity and Constraints

- **Unique:** customers.domain; tickets._id; agents.email; admins.email.
- **Indexes:** tickets (thread_id, company_id, customer_id, comments.message_id); customers (domain); agents (email); admins (email).
- **Validations:** Login requires non-empty email; ticket creation requires company_id, customer_id, subject, body; reply requires non-empty body; status in (Open, Pending, Closed).
- **Application-level:** Customer resolution creates company/contact when missing; ticket numbering (TCK-nnnn) and company/contact IDs (e.g. CMP-A001, CUS-A001) generated in sequence.

3.4. INTERFACE DESIGN AND PROCEDURAL DESIGN

3.4.1. User Interface Design

- **Login:** Single page with email and password fields; submit to POST /login; errors via flash; redirect to ticket list on success.
- **Ticket list:** Sidebar (All tickets, Open, Create ticket), main area with filters (status dropdown, domain dropdown), table/cards with ticket id, subject, status, priority, domain/company, updated time; link to detail.
- **Ticket detail:** Header (id, subject, status, priority, issue, domain/company/customer); chronological conversation (customer vs support, avatars and timestamps); reply box with textarea, “Suggest reply” button (AJAX), “Send reply” button; status change dropdown and submit.

- **Create ticket:** Form: customer dropdown (company — contact), subject, issue, priority, initial message; submit creates ticket and redirects to detail.
- **Layout:** Base template with sidebar, flash messages, and main content block; responsive styling (e.g. DM Sans, neutral palette, accent for primary actions).

3.4.2. Application Flow / Class Diagram

Reply suggestion flow: (1) User clicks “Suggest reply” on ticket detail. (2) Backend loads ticket and comments, builds conversation text. (3) Similar-ticket search: vectorise subject + customer messages, score against recent tickets with support replies, return top match. (4) If no similar or disabled: optionally run Phase 2 predictor (issue, priority, sentiment); call Gemini with conversation (and sentiment) and issue/priority; on failure or no API key, select template by issue. (5) Return suggested text and source (similar_ticket / gemini / template) to front end; user can edit and send.

Process flow diagram and/or class diagram (e.g. Flask app, EmailPipeline, Customer-Resolver, TicketManager, ReplySuggester, MLPredictor) can be drawn in a drawing canvas as per guidelines.

Fig. 3.3 — Application flow for reply suggestion

3.5. REPORTS DESIGN

- **Ticket list (screen):** Inputs: optional status, optional domain. Output: table/cards with ticket id, subject, status, priority, company/domain, updated_at.
- **Ticket detail (screen):** Input: ticket_id. Output: full conversation and metadata.
- **Future report ideas:** Tickets by status/count, by domain, by date range; agent workload; resolution time—to be implemented as separate report pages or exports.

4. IMPLEMENTATION

This chapter describes implementation approach, coding standards, important code details, and screen shots.

4.1. IMPLEMENTATION APPROACHES

Implementation followed an iterative approach: (1) Set up Flask app and MongoDB connection; define models and indexes. (2) Implement email pipeline (fetch, parse, customer resolution, ticket create/update, ack). (3) Implement web auth and ticket list/detail and reply send. (4) Implement reply suggestion (similar ticket, Gemini, template). (5) Add manual ticket creation and status update. (6) Integrate optional Phase

2 ML (load models, predict, enrich Gemini context). (7) UI polish and testing. Code is organised into `app.py` (routes and suggestion logic), `database.py` (MongoDB client and collections), `email_service.py` (IMAP/SMTP and pipeline), `models.py` (document structures), and `phase_2/sentiment_analysis.py` (ML classifier).

4.2. CODING STANDARD

- **Language:** Python 3.
- **Style:** PEP 8-oriented; meaningful names; functions kept focused.
- **Structure:** Imports at top; no inline imports except optional Phase 2 loader.
- **Security:** Passwords hashed (e.g. `werkzeug.security`); secrets and DB credentials in environment variables (`.env`).
- **Logging:** Standard logging for errors and key operations in email pipeline and app.
- **Configuration:** Environment variables for `FLASK_SECRET_KEY`, `GEMINI_API_KEY`, MongoDB URL/credentials, email credentials, `EMAIL_POLL_INTERVAL`, `AUTO_REPLY_ENABLED`, `TICKET_MODEL_PREFIX`.

4.3. CODING DETAILS

Customer resolution (`email_service.py`): The function `resolve_and_update_customer(sender_email)` first validates that the sender email contains an “@” and extracts the domain with `domain = sender_email.split("@")[1].lower()`. It then queries `customers.find_one({"domain": domain})`. If no customer document exists, it generates new IDs using `get_next_sequence("CMP", customers)` and `get_next_sequence("CUS", customers)`, creates a new document with `_id`, `company_name` (derived from domain), `domain`, `created_at`, and a single contact in the `contacts` array, and inserts it. If the company exists, it iterates over the `contacts` array; if any contact’s email matches the sender, it returns that `company_id` and `customer_id`. Otherwise it generates a new contact ID, creates a new contact object, and uses `customers.update_one` with `$push` to append the contact. The function returns `(company_id, customer_id)` for use in ticket creation or update.

Ticket threading (`email_service.py`): The function `resolve_or_create_ticket(email_data)` first normalises In-Reply-To and References into lists of message IDs via `parse_message_ids()`. It then loops through these IDs and checks (1) `tickets.find_one({"comments.message_id": ref})` and (2) `tickets.find_one({"ack_message_id": ref})`. If either finds a ticket, it returns that ticket’s `_id` and `is_new=False`. If not, it checks `email_data.get("thrid")` (Gmail thread ID) with `tickets.find_one({"thread_id": thrid})`. Then it uses a regex on the subject to find a pattern like TCK-0001 and looks up that ticket. If no match is found, it calls `get_next_ticket_number()` (which

finds the latest ticket `_id`, parses the number, and returns e.g. TCK-0002) and returns (new_ticket_id, True). The function `persist_ticket_and_message()` then either inserts a new ticket document with the first comment or, for an existing ticket, checks for duplicate message_id and uses `$push` to append the comment and `$set` to update `updated_at`.

Reply suggestion (app.py): The route `/tickets/<ticket_id>/suggest-reply` loads the ticket and builds conversation text with `_build_conversation_text(ticket, comments)` (subject plus “Customer:” / “Support:” lines). It then calls `_find_similar_ticket_replies(current_ticket_id, limit=300, top_k=3, min_score=0.42)`, which builds query text from subject and customer comments only, fetches recent tickets that have at least one support reply, and for each computes a similarity score as $0.7 * \text{SequenceMatcher.ratio}() + 0.3 * \text{Jaccard}(\text{token overlap})$. Tickets with score ≥ 0.42 are sorted by score and the top reply text is returned. If similar replies are found, the response JSON includes `suggested_reply`, source “similar_ticket”, and `similarity_score`. Otherwise the code optionally loads the Phase 2 analyzer with `_get_sentiment_analyzer()`; if loaded, it calls `analyzer.predict(conversation_text)` and uses the returned `issue_type`, `priority`, and `sentiment` to enrich the context (e.g. prepending “Customer sentiment: {sent}” to the conversation). Then `_ai_suggest_reply(conversation_text, issue=..., priority=...)` is called: it configures the Gemini API with the key from the environment, builds a prompt asking for a professional support reply with acknowledgment, body, and closing, and calls `model.generate_content()`. If that succeeds, the response is returned with source “gemini”. If it fails (no key, quota, or error), `_template_suggest_reply(ticket, customer_info)` is used, which selects a canned message by ticket issue (Technical, Billing, Account, Fraud, General Inquiry) and returns it with source “template”.

Sending reply (app.py): The route `/tickets/<ticket_id>/reply` (POST) loads the ticket and validates that the form body is non-empty. It retrieves the customer email via `get_customer_email_for_ticket(ticket)` (which looks up the company by `ticket.company_id` and finds the contact matching `ticket.customer_id`). If no email is found, it flashes an error and redirects. Otherwise it builds In-Reply-To from the last comment’s `message_id` and References from all comment `message_ids` (or from `ack_message_id` if no comments). It calls `send_support_reply(ticket_id, to_email, body, in_reply_to=..., references=...)` in `email_service`, which creates a MIMEMultipart message with the correct Subject (Re: Support Request [TCK-xxxx]), sets In-Reply-To and References for threading, and sends via SMTP. On success, the returned `message_id` and `body` are used to create a support comment object; `tickets.update_one` with `$push` adds this comment and `$set` updates `updated_at`. The user is redirected to the ticket detail with a success message.

4.4. SCREEN SHOTS

(Insert actual screen shots here; figures numbered per chapter, e.g. Fig. 4.1, 4.2, 4.3.)

- **Fig. 4.1 — Login screen:** Email and password fields, submit button.

- **Fig. 4.2 — Ticket list screen:** Sidebar, status/domain filters, list of tickets with id, subject, status, priority, domain, date.
 - **Fig. 4.3 — Ticket detail and reply screen:** Conversation thread, reply textarea, “Suggest reply” and “Send reply” buttons, status dropdown.
-

5. TESTING

This chapter documents test cases, testing approaches, and test reports.

5.1. TEST CASES

Login:

- TC-L1: Valid admin email and password → redirect to ticket list, session set.
- TC-L2: Valid agent email and password → redirect to ticket list, session set.
- TC-L3: Invalid password → error message, no redirect.
- TC-L4: Unknown email → error message.
- TC-L5: First-time setup (no users in DB) → first login as admin succeeds.

Ticket list and filters:

- TC-T1: Open /tickets with session → list of tickets, sorted by updated_at.
- TC-T2: Filter by status=Open → only Open tickets.
- TC-T3: Filter by domain → only tickets for that company domain.

Reply suggestion:

- TC-S1: Ticket with similar past ticket → suggestion from similar_ticket.
- TC-S2: No similar, Gemini configured → suggestion from gemini.
- TC-S3: Gemini unavailable or no key → suggestion from template.
- TC-S4: Phase 2 models loaded → prediction used in Gemini context.

Reply send:

- TC-R1: Valid body and customer email → email sent, comment stored, success message.
- TC-R2: Missing customer email → error message.
- TC-R3: Empty body → validation error.

Status update:

- TC-U1: Valid status (Open/Pending/Closed) → ticket updated, success message.
- TC-U2: Invalid status → error.

5.2. TESTING APPROACHES

- **Unit testing:** Functions such as `normalize_msg_id`, `parse_message_ids`, `get_body`, `extract_sender_email`, `_normalize_text_for_similarity`, `_similarity_score` can be unit-tested with fixed inputs.

- **Integration testing:** Email pipeline with a test mailbox; web app with test MongoDB; reply suggestion with mock or real Gemini.
- **Manual testing:** End-to-end: send email → run pipeline → see ticket in list → open detail → suggest reply → send reply → verify email and stored comment; login, filters, create ticket, status change.

5.3. TEST REPORTS

(Fill with actual results; sample format below.)

Test ID	Input / Scenario	Expected	Result
TC-L1	Admin credentials	Redirect, session	Pass
TC-L3	Wrong password	Error message	Pass
TC-T2	status=Open	Only Open tickets	Pass
TC-S2	Gemini key set, no similar	Gemini reply	Pass
TC-R1	Valid reply body	Email sent, comment saved	Pass

Table 5.x — Sample test report

6. CONCLUSION

6.1. DESIGN AND IMPLEMENTATION ISSUES

Design issues: Requirements clarity for threading (multiple match strategies) and reply suggestion tiers; balancing similarity threshold and Gemini fallback; optional Phase 2 dependency without breaking main app when models are missing.

Implementation issues: Email provider rate limits and IMAP behaviour across clients; timezone handling for comment ordering; ensuring In-Reply-To/References on sent emails so customer replies thread correctly; handling first-time DB with no admins/agents (bootstrap login).

6.2. ADVANTAGES AND LIMITATIONS

Advantages: Automated ticket creation and threading from email; customer and domain tagging; single dashboard for agents; reply suggestion reduces time and standardises quality; three-tier suggestion (similar, AI, template) keeps functionality even without API or ML; optional ML enriches context when available.

Limitations: Email-only ingestion; single-mailbox pipeline; no built-in assignment or SLA; Gemini dependency for AI suggestion (key and quota); Phase 2 classifier depends on training data quality and label alignment with production issue types.

6.3. FUTURE SCOPE OF THE PROJECT

- Add more channels (web form, chat) and unify in same ticket model.
 - Agent assignment and workload balancing.
 - SLA and escalation rules.
 - Rich analytics: resolution time, volume by domain/issue, sentiment trends.
 - Stronger Phase 2 integration: retraining pipeline, confidence thresholds, and A/B testing of suggestions.
 - Multi-tenancy and per-tenant configuration.
-

APPENDICES

APPENDIX A — USER MANUAL

A.1 Setup

- Install Python 3.9+, MongoDB, and pip dependencies (Flask, pymongo, python-dotenv, werkzeug, optional google-generativeai).
- Copy `.env.example` to `.env`; set `FLASK_SECRET_KEY`, MongoDB URL/credentials, `EMAIL_ADDRESS`, `EMAIL_PASSWORD`, optional `GEMINI_API_KEY`, `EMAIL_POLL_INTERVAL`, `AUTO_REPLY_ENABLED`.
- Run `create_indexes.py` to create MongoDB indexes.
- Optionally run `seed_database.py` to add seed tickets.
- For Phase 2: train `sentiment_analysis.py` to produce model files; set `TICKET_MODEL_PREFIX` if needed.

A.2 Running the system

- Start Flask: `python app.py` (default port 5005).
- Run email pipeline (separate process): `python email_service.py` (polls every `CHECK_INTERVAL` seconds).
- Open browser to `http://localhost:5005`; log in with an agent or admin account (or first user if DB empty).

A.3 Using the dashboard

- **Login:** Enter email and password; click login.
- **Ticket list:** Use “All tickets”, “Open”, or “Create ticket” in sidebar; filter by status and domain.
- **Ticket detail:** Click a ticket; read conversation; click “Suggest reply” to get text (edit if needed); type or paste reply, click “Send reply”; change status and submit if desired.
- **Create ticket:** Choose customer (company — contact), enter subject, issue, priority, and initial message; submit.

A.4 Phase 2 (optional)

- Place trained model files (e.g. ticket_model_issue.json, ticket_model_priority.json, ticket_model_sentiment.json, ticket_model_vectorizer.pkl, ticket_model_encoders.pkl) in project root or phase_2/; app loads them automatically when suggest-reply is used.

APPENDIX B — ADDITIONAL TABLES / CONFIGURATION

B.1 Environment variables

FLASK_SECRET_KEY, GEMINI_API_KEY, EMAIL_ADDRESS, EMAIL_PASSWORD, EMAIL_POLL_INTERVAL, AUTO_REPLY_ENABLED, TICKET_MODEL_PREFIX, SEED_TICKET_COUNT, MongoDB host/port/username/password/authSource.

B.2 Issue types (email_service detect_issue)

Login Problem, Payment Issue, Account Issue, Bug Report, Feature Request, General Inquiry.

B.3 Priority rule (email pipeline)

Payment Issue → High; others → Medium (manual creation allows High/Medium/Low).

B.4 Phase 2 training

The Phase 2 classifier is trained on a CSV with columns such as Ticket_Subject, Ticket_Description, Issue_Category, Priority_Level. The script phase_2/sentiment_analysis.py combines subject and description, preprocesses text (lowercase, strip “Hi Support;”, normalise spaces), and derives Sentiment from keywords (Urgent, Negative, Positive, Neutral). It uses LabelEncoder for issue, priority, and sentiment; TF-IDF (max_features=5000, ngram_range=(1,3)); and XGBoost for each task. Models are saved as ticket_model_issue.json, ticket_model_priority.json, ticket_model_sentiment.json, ticket_model_vectorizer.pkl, ticket_model_encoders.pkl. The main app looks for these in the project root or phase_2/ and loads them when the suggest-reply endpoint is first used.

APPENDIX C — PLANNING AND DESIGN TOPICS

C.1 Gantt chart / timeline (representative)

- Weeks 1–2: Requirements and design (problem definition, DFD, ER, architecture).
- Weeks 3–4: Database and email pipeline (MongoDB schema, IMAP/SMTP, customer resolution, ticket create/thread).
- Weeks 5–6: Web application (Flask routes, auth, ticket list/detail, reply send).
- Weeks 7–8: Reply suggestion (similar ticket, Gemini integration, templates).
- Weeks 9–10: Phase 2 ML (dataset, training, integration) and testing.
- Weeks 11–12: Documentation, report, and demonstration.

C.2 Design considerations

- **Health and safety:** The system is a software product with no direct physical risk; use of standard office ergonomics and display equipment applies for operators.
- **Security:** Passwords are hashed; credentials and API keys are in environment variables; session management restricts access to authenticated users only.

- **Usability:** The UI uses a clear sidebar, filters, and a single reply box with a suggest button to minimise steps for agents.

REFERENCES

- [1] Grinberg, Miguel. *Flask Web Development: Developing Web Applications with Python*. 2nd ed. Sebastopol: O'Reilly Media, 2018.
 - [2] MongoDB, Inc. "MongoDB Manual." 2024. 12 Mar. 2025 <https://www.mongodb.com/docs/manual/>.
 - [3] Python Software Foundation. "Python 3.9 Documentation." 2024. 12 Mar. 2025 <https://docs.python.org/3/>.
 - [4] Google. "Generative AI (Gemini API)." 2024. 12 Mar. 2025 <https://ai.google.dev/docs>.
 - [5] Chen, Tianqi, and Carlos Guestrin. "XGBoost: A Scalable Tree Boosting System." *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York: ACM, 2016. 785-794.
 - [6] Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research* 12 (2011): 2825-2830.
 - [7] Pressman, Roger S. *Software Engineering: A Practitioner's Approach*. 7th ed. New York: McGraw-Hill Education, 2010.
 - [8] Internet Engineering Task Force. "RFC 5322 - Internet Message Format." 2008. 12 Mar. 2025 <https://tools.ietf.org/html/rfc5322>.
-

End of Report